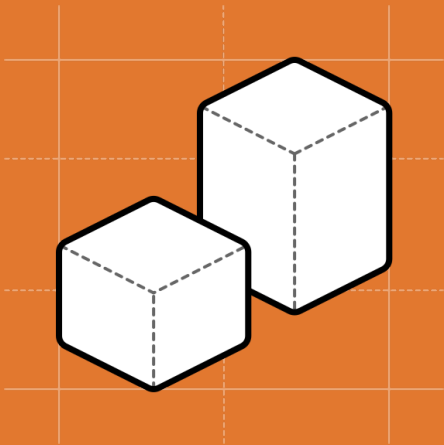


The Retention Protocol

Turn the first win into a habit, and the habit into mastery, with the agent leveling them up.



What's inside:

- The Level Design model: why retention is Level 2 of a game, and how the loop keeps a user coming back.
- The one repeated action a habit is built on, and how the agent rewards it, calibrates it, and always names the next move.
- Why friction is a feature, and how the agent shifts from doing the work to handing over the controls.

Contents

–	Introduction	03
–	The game: Level Design for software	04
–	Summary cheat sheet	05
01	Why users drift	06
02	The loop is the engine	07
03	Progress that means something	08
04	Reward like a game, never punish like a form	09
05	Always a next move, never a wall	11
06	Friction is a feature, bumpers not guardrails	13
–	The Retention Protocol	15
	The fill-in worksheet	
–	Run this with your favorite LLM	17

The level where the real money leaks

START HERE

Almost everyone who works on onboarding obsesses over the first few minutes. The signup, the empty screen, the first win. It's the part you can see, so it's the part teams pour their energy into. And then the curve falls off a cliff around day 30, and nobody can quite say why.

I'm Bal. I've spent about twenty years across marketing and product in B2B SaaS, working with teams at companies like Google, Microsoft, and Intel. Titles changed over the years; the two disciplines didn't. For most of that time I was designing onboarding without calling it that, the whole stretch between "just signed up" and "I can't work without this now." And the thing I kept seeing is that the first session is rarely where products lose people for good. They lose them later, quietly, after a good start.

Here's the pattern. A user signs up, gets a real first win, and you celebrate the activation number. Then week two comes and the product has nothing new to say to them. They've seen the trick. The thing that was exciting on Monday is routine by Friday, and routine with nothing past it is boredom. They don't rage-quit. They just open it less, then not at all, and three weeks later they're a churn statistic that your activation funnel never warned you about.

That slow leak is bigger than the fast one. Activation stops the people who bounce on day one. Retention is the much larger group who gave you a real shot and drifted anyway, and winning them back is most of the money in a subscription business. This is Level 2 of the game, and it's the level most products skip.

One word on what I mean by the agent, because it runs through this whole protocol. The agent is the AI built into your product, the one that can talk to the user and actually do things for them, not a chatbot bolted on the side of an unchanged product. If you're building an agent-led product, that AI is already the main surface the user touches. If you're a more traditional product just now adding AI, it's the layer you're putting in. Either way the move is the same: that AI doubles as the onboarding, the same companion on day one and in month six, so there's no separate onboarding flow to graduate out of.

This protocol stands on its own. If it's the only one you ever read, you'll still get the whole model.

The goal, in one line: keep raising the bar in step with the user's growing skill, so the win becomes a habit and the habit becomes mastery, and the product becomes part of how they work.



The model everything runs on

The game: Level Design for software

RETENTION IS LEVEL 2 · THE LOOP IS THE ENGINE · THE AGENT KEEPS RAISING THE BAR

Before the tactics, the model the whole stack runs on. If you only take one idea from this protocol, take this one.

Think of your product as a game, and your user as a player who starts as a stranger and, if things go right, becomes a power user who can't imagine working any other way. Getting them there isn't a phase that ends after week one. It's a series of levels they grow through, and the good ones never stop teaching. Think of a deep role-playing game, the kind people sink a hundred hours into, where each level hands the player a skill they'll need for the next, and new abilities open up as they master the old ones. Mastery is the destination, not the exit.

That game has three levels, and they're the three protocols in this stack:

- **Level 1, Activation.** The first real win. A stranger reaches a result that makes them think "okay, this works."
- **Level 2, Retention.** The win becomes a habit. They come back, get better, and the product opens up as they do. This protocol.
- **Level 3, Expansion.** They hit the edges of what they've got, reach for more power, and start bringing other people in.

What carries a player from one moment to the next is a loop: a **cue** that pulls them in, an **action** they take, and a **reward** that pays off the action and sets up the next cue. Cue, action, reward, next cue. In activation that loop fires once, to land the first win. In retention that same loop is the whole game: a retained user is just someone running a loop you designed, over and over, because each turn pays off and points at the next.

There's a reason a phase-based view of onboarding falls apart here. A player stays engaged only when the challenge keeps pace with their skill. Too hard, they get anxious and quit; too easy, they get bored and drift. The sweet spot is a narrow band between the two, and it moves: as the player improves, yesterday's challenge becomes today's boredom. A product that teaches everything in week one then goes silent has parked the user in the boredom half of that band. That's the day-30 drift, and it's why retention is a challenge-calibration problem, not a missing-feature one.

And in agent-led software, the game finally has a companion that can keep up. A hardcoded difficulty curve has to guess where the average player is. The agent can see where this specific user is: what they use, what they repeat, what they've never touched. It can raise the bar for the person in front of it, right when their skill makes the old level boring. That's real per-user calibration, the thing static products could never do.

Everything that follows is how to design Level 2 so the agent can run it.

The whole protocol, one page

Read the rest for the why, or jump to the Retention Protocol on page 15.

The game

- Retention is Level 2: the first win turning into a habit, then mastery.
- A retained user is someone running a loop you designed, over and over, because each turn pays off.
- The challenge has to keep pace with the user's growing skill, or they drift. Retention is calibration, not features.

The drift

- Day-30 churn usually isn't a broken feature. It's the product going quiet after a good start, parking a now-competent user in boredom.
- The agent can see where each user actually is and raise the bar per person.

The loop

- Find the one core action a retained user repeats, and build the loop around it: cue, action, reward, next cue.
- The reward has to be real value in the user's terms, not a "nice job." The return cue, timed and personalized by the agent, is what brings them back.

Progress and reward

- Progress only motivates if it ties to something the user wants. A setup bar is theater; "two more fields and your forecast is trustworthy" is real.
- Unlock real capability as they grow; confetti wears off. And every red error is a stick you could restate as a carrot.

→ The next move

- Never one mandatory path. Offer a few live fronts so being stuck on one doesn't end the session, and let the agent always name one unblocked next move.
- Do it for them, then with them, then watch them do it. A product that does everything makes a dependent, not a power user.



01

Why users drift

NOTHING BROKE. THE PRODUCT JUST WENT QUIET.

When a user churns at day 30, the instinct is to go looking for what broke. A bug, a missing feature, a competitor with a better price. Sometimes that's it. But most day-30 drift isn't a failure of the product, it's a failure of pacing. The user got their first win, learned the one thing the product taught them, and then the product ran out of things to say. Nothing broke. It just went quiet, and quiet is where habits go to die.

To see why, picture the band a person has to stay in to stay engaged. On one side is anxiety: the task is too hard, they feel lost, they quit. On the other side is boredom: the task is too easy, there's nothing left to figure out, they drift. Staying engaged means living in the narrow channel between those two. The catch is that the channel moves. As someone gets better, the thing that challenged them last week is routine this week, so the challenge has to rise to keep pace. A good game does this for hundreds of hours. Most software does it for about one session.

That's the real shape of the day-30 problem. The first-run experience is tuned to take a nervous beginner and get them a win, which is exactly right for day one. But the product never re-tunes. It keeps treating a now-competent user like a beginner, showing the same prompts, offering the same shallow win, and a competent user reads that as the product having nothing more for them. Retention isn't a feature you forgot to build. It's a difficulty curve you never drew.

The clearest proof that this is solvable lives in the games people never finish quitting. Take a long-running online role-playing game like World of Warcraft, where players stay for years. It never graduates you. Every new area assumes the skills the last one taught and demands one more, so there's always a next thing slightly beyond your current reach. The game is, in effect, onboarding you the entire time you play. It just never calls it that, and it never stops.

Static software can't really do this, because it can't tell a beginner from an expert; it shows everyone the same screens. An agent can. It can see what this user has actually done: which features they lean on, which action they repeat every day, which half of the product they've never opened. That's the raw material of calibration. When the agent notices someone has run the same report by hand three weeks running and says "want me to set this to run automatically and just message you when it's ready," it's doing exactly what a good game does, raising the challenge at the moment the old way got boring. The difference is it's doing it for one specific person, not an imagined average.

Stop asking what feature would stop the churn, and start asking where this user is on the curve and what the next challenge should be. Retention is the work of keeping the bar moving in step with the user.



02

The loop is the engine

CUE, ACTION, REWARD, NEXT CUE. THAT'S THE WHOLE GAME.

A habit isn't a feeling, it's a loop that runs often enough to become automatic. Four parts: a **cue** that pulls the user in, an **action** they take, a **reward** that pays the action off, and a **next cue** that the reward sets up. Get one turn of that loop to feel good and point at the next turn, and you don't have to motivate the user back; the loop does it. This is the single most important mechanic in retention, so it's worth building deliberately instead of hoping it emerges.

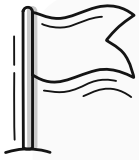
Start with the action, because everything else hangs off it. Find the one core action a retained user repeats. Not the impressive feature, not the thing your roadmap is proud of; the small thing your best users do over and over, the move that, if they stopped doing it, would mean they'd churned. For an analytics tool it might be checking one number every morning. For a writing tool, starting one draft. For a CRM, logging one deal update. If you can't name that action in a sentence, you don't yet know what habit you're trying to build, and you can't design a loop around a habit you haven't named.

Then make the action pay off inside the product, in the user's own terms. A reward isn't a "nice work" toast or a checkmark; it's the user getting something they actually wanted, and seeing that they got it. The payoff for checking the number is the number being genuinely useful that morning. The payoff for starting the draft is the draft being easier to finish than a blank page. The reward has to be real value, not a celebration of value, because users learn fast which is which and stop showing up for the confetti.

Then the return cue, which is the part most products leave to chance. Something has to bring the user back tomorrow, and "they'll remember us" is not a plan. The cue can be in the world (a morning routine the product slots into) or it can be something the product sends. The trap is the generic blast: the same "you have updates" email to everyone, which trains people to ignore it. A good cue is specific and timed to a real moment, the kind only a system that knows the user can send.

That last part is where an agent changes the math. A static product fires the same notification on a schedule. An agent can hold the cue until it's actually worth sending and make it about the user's own world: "your weekend numbers just came in and one of them is off in a way I think you'll want to see." That's not a reminder, it's a reason. The agent can also close the loop out loud, reporting the reward in plain language the moment the action lands, then naming the next cue in the same breath, which is the loop turning over on its own.

Name the one action your habit is built on, give it a reward that's real value and not a celebration, and design the cue that brings the user back instead of hoping they return. That loop is the engine of retention.



03

Progress that means something

REAL PROGRESS TIES TO WHAT THE USER WANTS. THEATER IS JUST A NUMBER GOING UP.

Once the loop is running, the natural next move is to show the user they're getting somewhere. Progress bars, completion percentages, streaks, levels. The instinct is right, because visible progress is genuinely motivating. But there's a test most progress fails, and getting it wrong is worse than showing no progress at all, because it teaches the user that your signals don't mean anything.

The test comes from two old platform games, and it's the cleanest comparison in the medium. In one of them, the hero was a cat who collected balls of yarn. The yarn added points, the points did nothing you cared about, and players figured that out within minutes and stopped bothering to collect them. In the other, Sonic the Hedgehog, the hero collected gold rings, and the rings protected you: get hit while holding rings and you survive, get hit with none and you die. Players chased every single ring. Same basic mechanic, collect-the-thing, opposite result, and the only difference was whether the thing connected to something the player already wanted.

That gives us the test, and it's worth naming the two kinds so you can sort your own product by them. Call the kind that connects to something the user wants **real progress**, and call the kind that's just a number climbing for your benefit **progress theater**. A "profile 60% complete" bar is theater; it measures work the user does for you, dressed up as their achievement. "Add two more fields and your forecast gets accurate enough to actually trust" is real progress; the same request, but tied to something they came to the product for. The first kind users learn to ignore. The second they finish, because finishing buys them something real.

So audit your progress honestly. Walk through every bar, badge, streak, and percentage in the product and ask, for each one, what the user actually gets when it fills. If the answer is "we get cleaner data" or "they've used more of the product," that's theater, and it's training your users that your signals are noise. If the answer is a capability, an accuracy, an outcome they wanted, that's real progress. Cut the theater or rewire it to a real payoff. Progress theater doesn't just fail to motivate, it actively spends the trust you'll need when you show a signal that does matter.

The agent makes the real version cheap to do, because it can attach the payoff to the ask in the same sentence, every time, without you hard-coding a hundred different messages. "Connect your billing data and I can tell you which plan tier is quietly churning" is real progress stated on the fly. The agent never has to ask for a step it can't pay for, because it can always name the payoff in the user's terms at the moment it asks.

Visible progress works, but only when each step is real progress, tied to something the user already wants, and not theater that only benefits you. Audit what you've got, cut the theater, and let the agent state the real payoff every time it asks for a step.



04

Reward like a game, never punish like a form

EVERY RED ERROR IS A STICK YOU COULD HAVE MADE A CARROT

Games are generous in a very deliberate way, and software mostly isn't. A game rewards you constantly, and it's careful about how it punishes. Software tends to do the opposite: it's stingy with reward and quick to punish, and it calls the punishment "validation." Flipping that ratio is one of the highest-leverage moves in retention, because reward is what makes a user want the next turn of the loop, and punishment is what makes them quit it.

Start with reward, and be picky about which kind. The flashy ones, confetti, a celebration animation, a "great job" banner, are spectacle, and people acclimatize to spectacle fast; by the fifth time, the confetti is wallpaper. The rewards that last are the ones that change what the user can do: a new capability that opens up, a gate that lifts, a power that wasn't there before. "You've done this enough that I can now do it for you automatically" is a reward a user will work toward, because it's real and it compounds. Lead with those, and use the confetti sparingly, if at all.

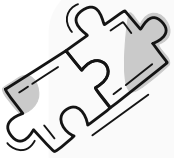
Now punishment, where most products quietly bleed users. The model worth stealing comes from the action role-playing game Diablo. An early version had a hunger mechanic that punished you: let your character go too long without eating and they'd weaken and eventually die, so you were constantly being penalized for forgetting a chore. It felt awful. So the designers flipped it. In the shipped game you never starve; instead, eating gives you a temporary boost. Exact same behavior being encouraged, eat regularly, but one version nags you with a stick and the other rewards you with a carrot. The behavior is identical; the feeling is opposite.

Your error states are full of hunger mechanics waiting to be flipped. A red "this field is required" is a stick. "Add your domain and your deliverability jumps, want me to walk you through it" is the same requirement as a carrot. Audit every error, every blocked state, every nag in the product and ask the Diablo question: can this stick be restated as a carrot? Most can. And the ones that genuinely can't, the hard stops, should at least never punish a user for something they couldn't have predicted or prevented, because a punishment that feels unfair is the kind of thing that makes someone stop engaging entirely.

There's a reward worth offering that doesn't look like a reward at first: a bolder choice. Alongside the safe, standard path, offer a riskier, more interesting one. The old arcade game Space Invaders is mostly a steady grind of shooting rows of aliens, except every so often a special saucer flies across the top, hard to hit and worth far more points, and chasing it is optional. That one risky target is most of why the hundredth wave still has any tension. In your product it looks like "I can set this up the standard way, or, riskier but more useful, I can try building it straight from your own data, which way do you want." The bold option is itself a reward; letting a competent user choose it is how they level up by choice instead of by force.

This is where the agent earns its keep, because it can do the flipping live. It fixes what it can, flags what it can't, and frames the gap as upside instead of failure: "this works as is; add a custom domain later and your deliverability goes up." One honest caution before you get clever with rewards: resist the urge to make them random for the dopamine, the slot-machine trick of unpredictable payouts. It works on people, which is exactly why it's the wrong move in a tool they trust with their business. Borrow the unlock structure from games. Skip the casino.

Lead with rewards that change what the user can do, restate every stick as a carrot with the Diablo test, offer a bold option as its own reward, and let the agent do the flipping in real time.



05

Always a next move, never a wall

STUCK ON ONE THING SHOULDN'T MEAN STUCK, PERIOD

A user who hits a wall and can't see a way around it leaves, and they rarely come back, because being stuck with no visible next move is the most discouraging state a product can put someone in. The fix isn't to remove every obstacle; it's to make sure there's always at least one move the user can make right now. Walls come from a single mandatory path. The cure is more than one path.

Most products onboard, and then keep operating, on a single line: do this, then this, then this. The problem is that any one of those steps can block, waiting on a teammate, an integration, a piece of data the user doesn't have yet, and the moment it blocks, the whole session is over. There's nothing else to do. Compare that to how a well-built game keeps several things open at once: a main quest, a couple of side quests, something to craft, somewhere to explore. Being stuck on the main quest never means being stuck, because there are three other worthwhile things to go do. Offer the same in your product: a few live fronts, not one. Import your data, or invite a teammate, or set up your first automation. Block one and the user still has somewhere to go.

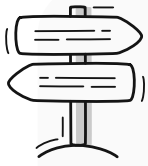
Those fronts should add up to something, though, or they're just busywork. The trick games use is to stack small goals into bigger ones into one clear peak: the small tasks feed a medium milestone, and the medium milestones feed the single thing the player is ultimately working toward. The user should be able to feel that the little move they make today is a step toward the big outcome they signed up for, not an errand. That sense of climbing toward a peak, while always having a small next step in hand, is what keeps someone in the product through the long middle where the novelty has worn off.

Holding all of that in your head, the long arc, the mid milestone, the next five minutes, is a lot to ask of a user, and most won't. This is the job the agent is built for. It can hold the whole pyramid so the user doesn't have to: it knows the outcome they're working toward, the milestone they're closing in on, and it can always surface one unblocked move right now. "The import's still waiting on your data team, so while that sits, want me to set up your first alert; it's two minutes and it's the other thing you said you wanted." There's never a wall, because the agent always has another live front to point at.

The deepest version of this move is what a great game character does, and it's the single most important thing an agent can learn. In the theme-park boat ride Pirates of the Caribbean, the enemy ships don't just fight; they're quietly choreographed to flee toward the next scene the designers want you to see, so the action keeps pulling you somewhere good while you feel like you're just steering your own boat. A good agent does the same thing: it serves the request in front of it, and it steers the user one level up at the same time. Answer the question, then point at the island they can't see yet. "Done. By the way, the thing you just did by hand, I can watch for that

automatically from now on, want me to." The user gets what they asked for and a nudge toward mastery, and it feels like their own idea, which is exactly the feeling that keeps them leveling.

Never run the user down a single path that can wall them. Offer a few live fronts that ladder up to the outcome they want, let the agent hold the whole arc, and let it steer the user up a level while it answers what they actually asked.



06

Friction is a feature, bumpers not guardrails

THE MENTOR CAN'T BE THE HERO

It's tempting to read this whole protocol as "remove all friction and do everything for the user." That's the wrong lesson, and following it produces a user who never actually learns the product and churns the moment a real demand shows up. Some friction is the point. The skill is telling the friction that teaches from the friction that just blocks, and keeping the first while killing the second.

Think of the difference as bumpers versus guardrails. A guardrail stops you cold; you hit it and you're done. A bumper, the kind in a bowling lane, keeps you in play; you bounce off it and keep rolling toward the pins. Good friction is a bumper. It's the small effort that makes the user actually understand what they're doing: a confirmation that makes them think for a second, a manual first pass before the automation takes over, a mistake they're allowed to make safely and learn from. Bad friction is a guardrail: a required field with no explanation, a dead end, a punishment for not knowing something the product never taught. Keep the bumpers. Tear out the guardrails.

The most valuable friction of all is a safe mistake. People learn systems by doing, and by getting it wrong in a place where wrong is cheap. A product that lets a user try something, watch it not work, and understand why, has taught them something no tooltip could. This is where the agent has a move static products never had: it can turn any risky action into a dry run, "want me to show you what this would do before I actually do it," and it can turn a failure into the lesson, "that didn't go through because the date field isn't mapped; here's what I'd try instead." Failure stops being a support ticket and becomes the curriculum.

Which brings up the real danger of an agent that's too helpful: it can rob the user of the friction that would have made them competent. The model to hold is the mentor in any good adventure story. The mentor equips the hero, trains the hero, and then stays out of the fight, because a mentor who fights every battle produces someone who can't fight at all. An agent that does everything forever builds a dependent, and dependents churn the instant the novelty fades, because they never developed the skill that would make leaving costly. The posture that works is do it for them on the first attempt, do it with them on the second, and watch them do it on the third. Confident advice with reasons, and the user's hands on the final call: "I'd go with weekly cohorts here, here's why, but it's your call."

And that points at the real measure of whether retention is working, which isn't a usage number at all. The question is whether the user changed how they work. There's a story game designers like to tell about a kids' online game called Toontown, where the only way to chat was to pick from a menu of polite, preset phrases, no free typing. A player who came in a trash-talker found, after a while, that his habits had softened, because the structure quietly rewired him without ever

announcing a goal. That's the bar for a product, too. A user who has genuinely changed how they work because of you has switching costs no contract can match, and they tell people, because they're proud of what they can now do. That user has finished Level 2. They're a habit-formed regular starting to push at the edges of what they've got, which is the exact moment Level 3, Expansion, begins: more power, higher tiers, and bringing other people in.

Keep the friction that teaches and cut the friction that only blocks, let the agent make failure safe and instructive, and have it hand over the controls instead of holding them forever. When the user has changed how they work, retention has done its job, and the next level is Expansion.



The worksheet

The Retention Protocol

DO IT YOURSELF, OR HAND IT TO YOUR FAVORITE LLM

Work through it for your product, or hand it to your favorite LLM along with this protocol and your product details and have it draft each answer with you, then sharpen. Either way you finish with a named habit loop, progress and rewards that mean something, and an agent that always has a next move.

STEP 1: NAME YOUR CORE REPEATED ACTION

The one small thing a retained user does over and over. If they stopped doing it, they've churned.

STEP 2: BUILD THE LOOP AROUND IT

Fill each corner. If you can't fill one, that's where the habit is leaking.

LOOP PART

FOR YOUR PRODUCT

Cue (what pulls them back)

Action (the core repeated move)

Reward (the real value, in their terms)

Next cue (what the reward sets up)

STEP 3: RUN THE PROGRESS AUDIT

List your progress elements (bars, streaks, badges, percentages) and mark each one. Theater gets cut or rewired; real progress stays.

PROGRESS ELEMENT

WHAT THE USER ACTUALLY GETS

REAL

THEATER

☐☐☐☐☐☐

STEP 4: FLIP THE STICKS TO CARROTS

List your error states, blocked states, and nags. For each, write the carrot version.

THE STICK (TODAY)

THE CARROT (RESTATED)

STEP 5: MAP THE NEXT MOVES SO THERE'S NEVER A WALL

The long-term outcome the user is working toward:

--

The mid-term milestone on the way there:

--

The 2-3 live fronts available at any time (so one block never ends the session):

STEP 6: WRITE THE AGENT'S LEVEL-UP NUDGE

The collusion line: the agent answers a real request, then steers the user one level up in the same breath.

When the user does _____ (a request),
the agent answers it and then offers _____ (the next level up).

All six filled in is your Level 2 design: a named habit loop with a reward that's real value, progress that ties to what the user wants, sticks restated as carrots, a path with no walls, and an agent that always names the next move. That's retention. Expansion is the next level.



The dual use

Run this with your favorite LLM

THE PROTOCOL IS THE EXPERT; YOUR AI ASSISTANT DOES THE LOCALIZING

You can read this protocol and do the work yourself. The faster move, and the one it's built for, is to hand the whole thing to your favorite LLM and design your retention with you. Attach this protocol, give it a few paragraphs about your product (what it is, who it's for, what a healthy active user does day to day), then work through these asks in order.

ASK YOUR LLM	WHAT YOU'RE GETTING	CH.
"Based on my product, what's the one core action a retained user repeats, and where on the difficulty curve do my users tend to stall?"	The habit your loop is built on, and where the drift starts.	01, 02
"Design the full loop around that action: the cue, the action, the in-product reward in the user's terms, and the return cue."	A complete habit loop with no missing corner.	02
"Audit my progress elements and tell me which are real progress and which are theater, then rewire or cut the theater."	Progress that motivates instead of training users to ignore it.	03
"Go through my error and blocked states and restate every stick as a carrot, and flag any that can't be."	A reward-heavy, fair version of the moments that usually make people quit.	04
"Map the long arc, the mid milestone, and 2-3 parallel fronts so my product never walls a user."	A no-wall structure the agent can always pull a next move from.	05
"Write the agent's level-up nudges: how it answers a real request and steers the user one level up in the same breath."	The collusion lines that turn helping into leveling.	05, 06

Push back on your AI assistant the way you would on a teammate. If a reward reads like confetti instead of real value, say so and make it try again. The model gives you the structure and the judgment; your taste decides what's actually right for your users. That last bit is the part that's yours.

Get them their first win, then keep raising the bar so the win becomes a habit and the habit becomes mastery. That's the level most products skip.

The Onboarding Loop Protocols

BAL SIEBER · RETENTION, LEVEL 02